

Introduction to RNN & LSTM



Introduction to RNN:-

RNN stands for Recurrent Neural Network.

It is a type of artificial neural network designed for sequential data processing.

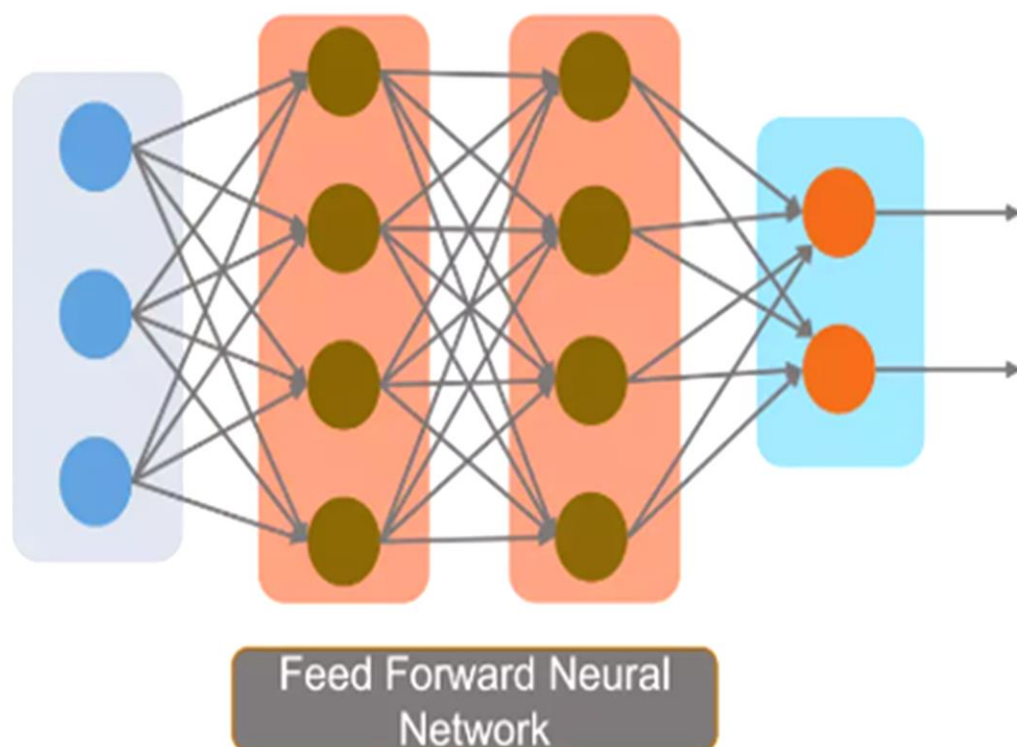
This neural network takes the output of the previous step and feed as the input of current step.

Unlike feedforward neural networks, which process input data in a single pass, RNNs have connections that form directed cycles, allowing them to exhibit temporal dynamic behavior. This architecture makes RNNs particularly well-suited for tasks involving sequences, such as natural language processing, speech recognition, time series prediction, and more.

In an RNN, each node (or neuron) in the network has a directed connection to itself, allowing it to maintain a memory of past inputs. This memory enables the network to capture dependencies and patterns over time. The basic computation in an RNN involves taking an input vector and a hidden state vector from the previous time step, applying a set of weights, and passing the result through a non-linear activation function to produce an output and an updated hidden state. This process is repeated for each time step in the sequence.

Why Recurrent Neural Network?

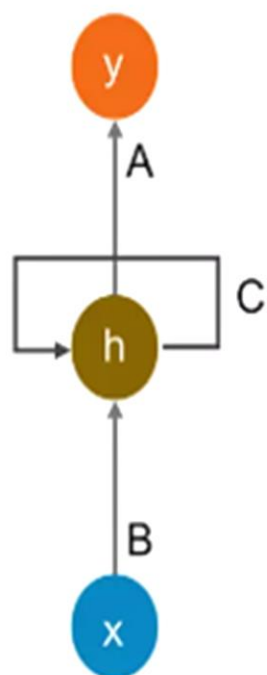
Issues in Feed Forward Neural Network



- 01 cannot handle sequential data
- 02 considers only the current input
- 03 cannot memorize previous inputs

Why Recurrent Neural Network?

Solution to Feed Forward Neural Network



Recurrent Neural
Network



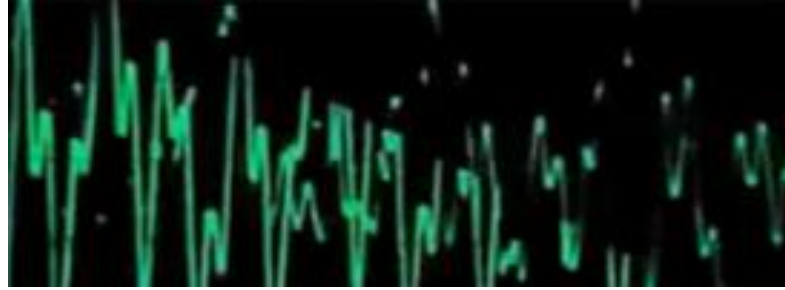
- 01 can handle sequential data
- 02 considers the current input and also the previously received inputs
- 03 can memorize previous inputs due to its internal memory

Applications of Recurrent Neural Network

1. Language Modelling and Generating Text
2. Speech Recognition
3. Machine Translation
4. Image Recognition, Face detection
5. Time series Forecasting

Examples of Sequence Data

Speech Recognition



He is a good boy

Music Generation



Sentiment Analysis

I don't like it



DNA Sequence Analysis

AGCCCCTGTGAG

“AGC**CCCTGTGAG**”

Language Translation

प्रकृति बहुत सुन्दर है

Nature is very beautiful

Video Activity Analysis



Running

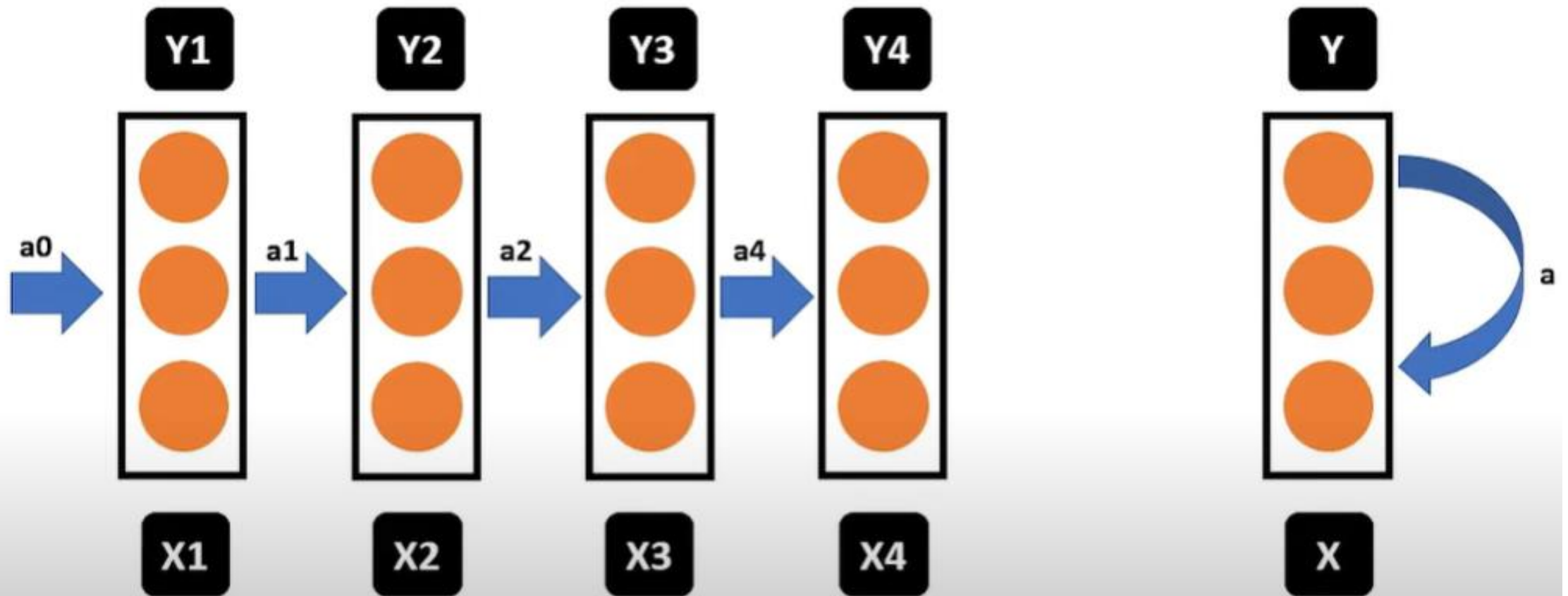
Advantages of Recurrent Neural Network

- ❑ **Ability to Capture Temporal Dependencies:** RNNs are designed to handle sequential data by maintaining a memory of past inputs through recurrent connections. This enables them to capture temporal dependencies in the data, making them well-suited for tasks where the order of input elements matters, such as time series prediction, natural language processing, and speech recognition.
- ❑ **Variable Length Input Sequences:** Unlike traditional neural networks, which require fixed-size inputs, RNNs can process sequences of variable lengths. This flexibility is crucial in tasks where the length of input sequences may vary, such as text processing and speech recognition.
- ❑ **Shared Parameters Across Time Steps:** RNNs share the same set of parameters across different time steps, allowing them to efficiently process sequences of arbitrary length. This parameter sharing enables RNNs to generalize well to inputs of varying lengths and reduces the computational cost compared to models with separate parameters for each time step.

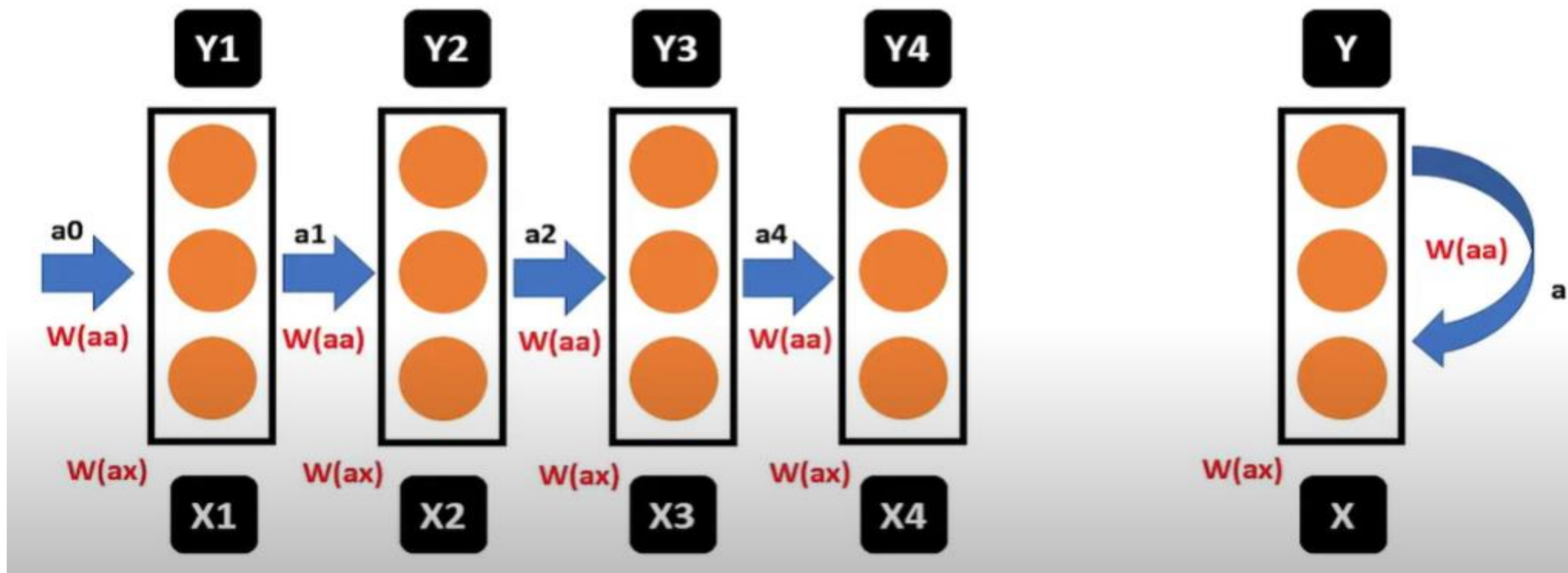
- ❑ **Implicit Feature Extraction:** RNNs can automatically extract relevant features from sequential data without the need for handcrafted feature engineering. This is particularly advantageous in tasks involving complex and high-dimensional sequential data, where manual feature extraction may be challenging or impractical.
- ❑ **Ability to Handle Time-Series Data:** RNNs are effective for modeling and predicting time-series data due to their ability to capture temporal dependencies. They can learn patterns and trends in time-series data, making them valuable in applications such as financial forecasting, weather prediction, and sensor data analysis.
- ❑ **Flexibility in Architectural Design:** RNNs offer flexibility in architectural design, allowing for variations such as Bidirectional RNNs, LSTM (Long Short-Term Memory) networks, GRU (Gated Recurrent Unit) networks, and hierarchical RNNs. These variations enable researchers and practitioners to choose the most suitable architecture for their specific tasks and datasets.
- ❑ **Interpretability:** RNNs provide some level of interpretability by capturing the sequential structure of the data. By analyzing the activations and weights of the network, researchers can gain insights into how the model processes and represents sequential information, aiding in understanding the underlying patterns in the data.

How does RNN work?

The Recurrent Neural Network consists of multiple fixed activation function units, one for each time step. Each unit has an internal state which is called the hidden state of the unit. This hidden state signifies the past knowledge that the network currently holds at a given time step. This hidden state is updated at every time step to signify the change in the knowledge of the network about the past.



Forward Propagation



$$a0 = \begin{matrix} \rightarrow \\ 0 \end{matrix}$$

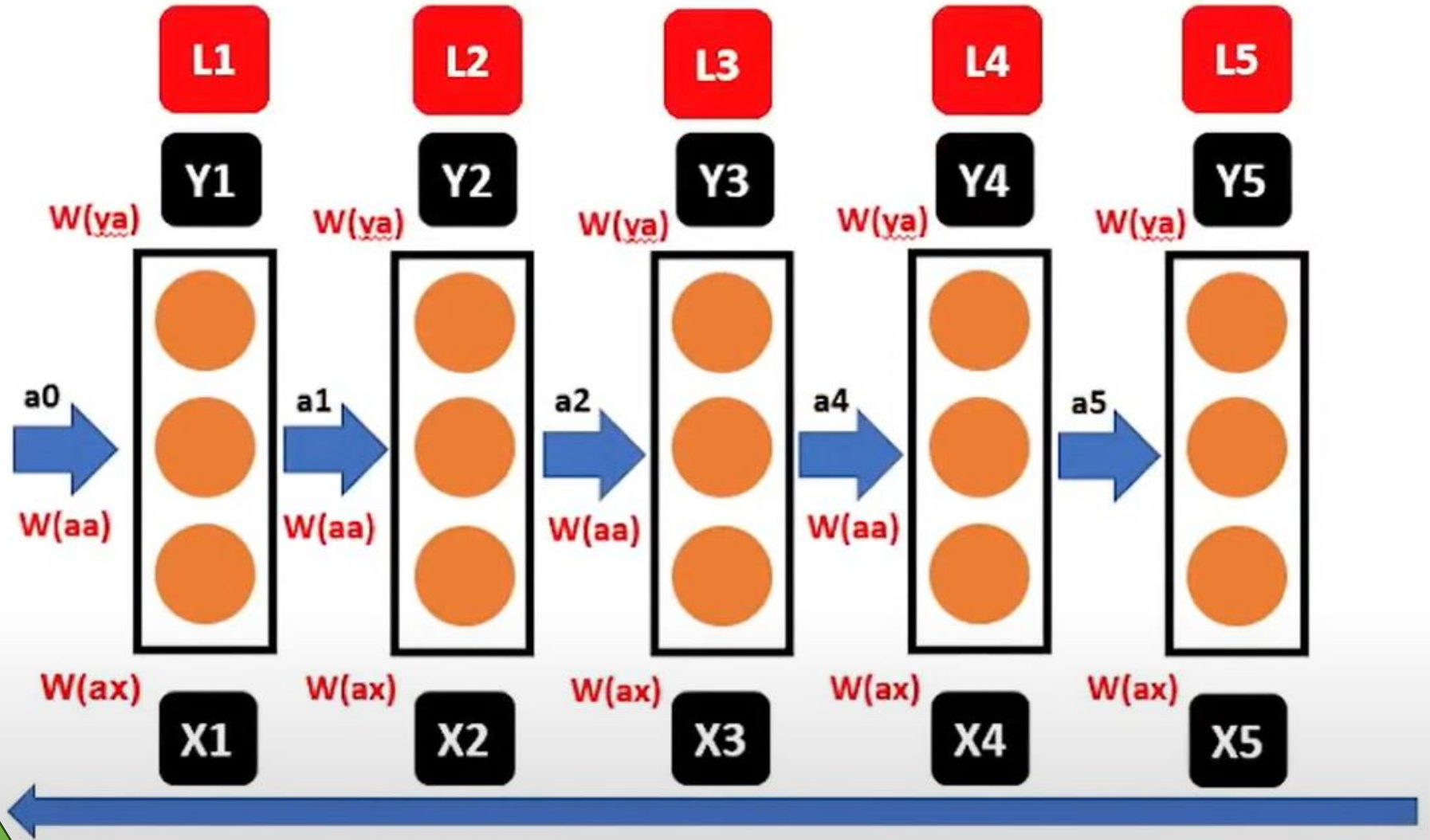
$$a1 = g(W(aa)*a0 + W(ax)*X1 + b(a))$$

← Tanh/Relu

$$Y1 = g(W(ya)*a1 + b(y))$$

← Sigmoid/Softmax

Backward Propagation



Entropy Loss
Formula

$$\text{Loss} = -\frac{1}{\text{output size}} \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i + (1 - y_i) \cdot \log (1 - \hat{y}_i)$$

Types Of RNN

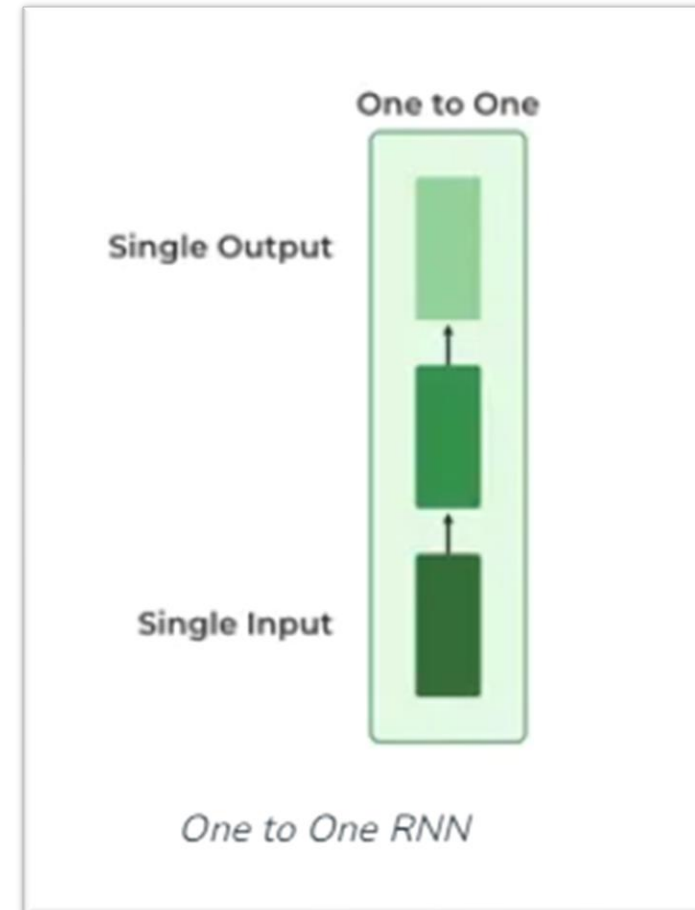
There are four types of RNNs based on the number of inputs and outputs in the network.

1. One to One
2. One to Many
3. Many to One
4. Many to Many

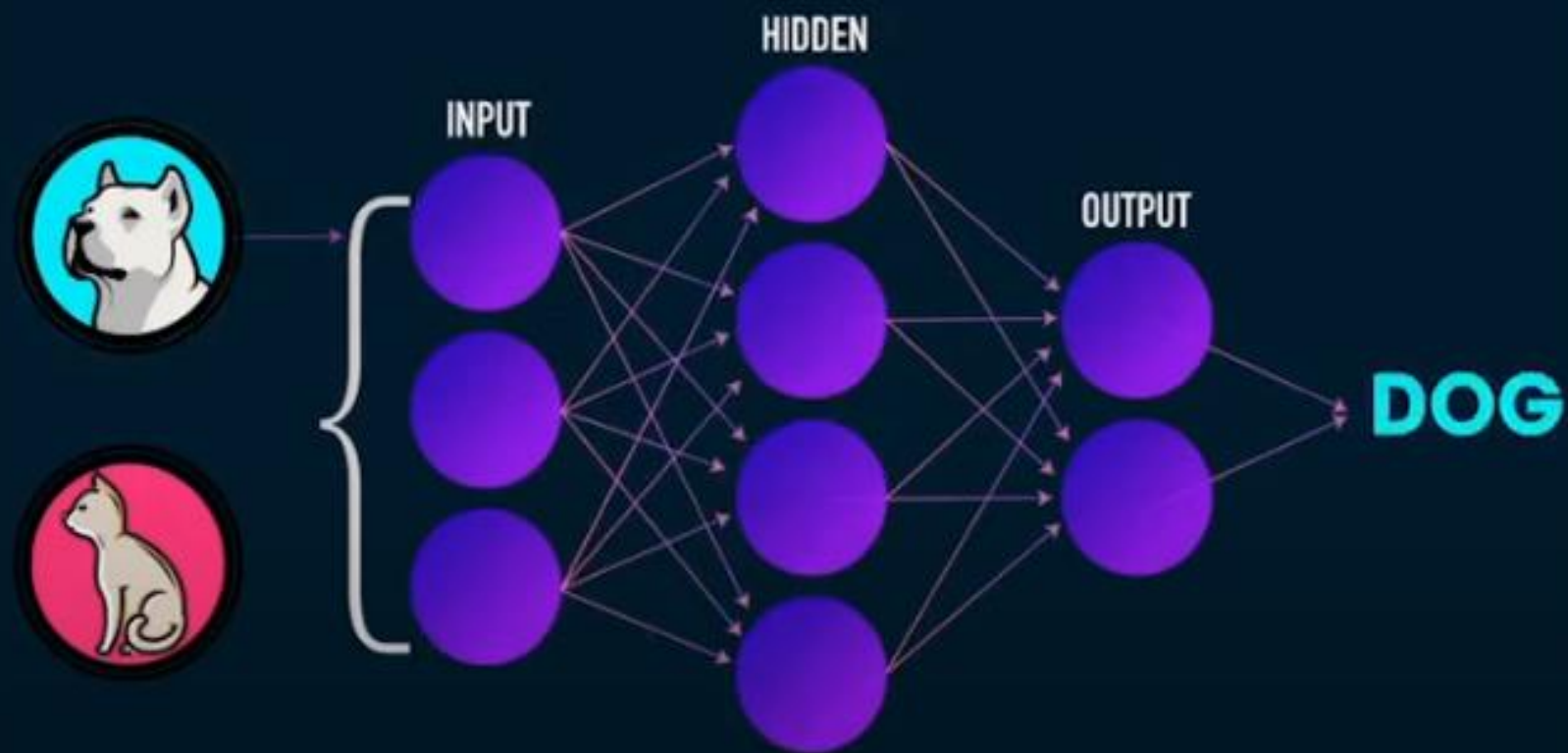
One to One

This type of RNN behaves the same as any simple Neural network.

In this Neural network, there is only one input and one output.

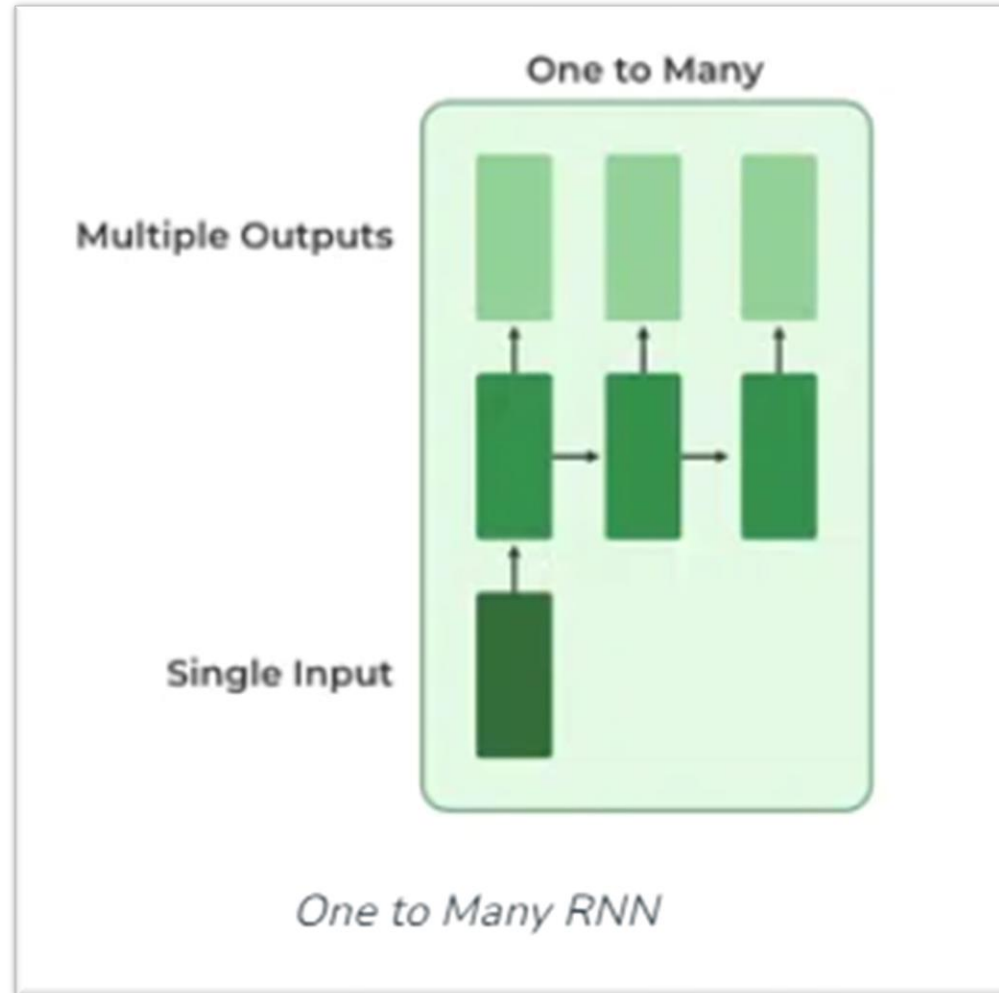


One to One

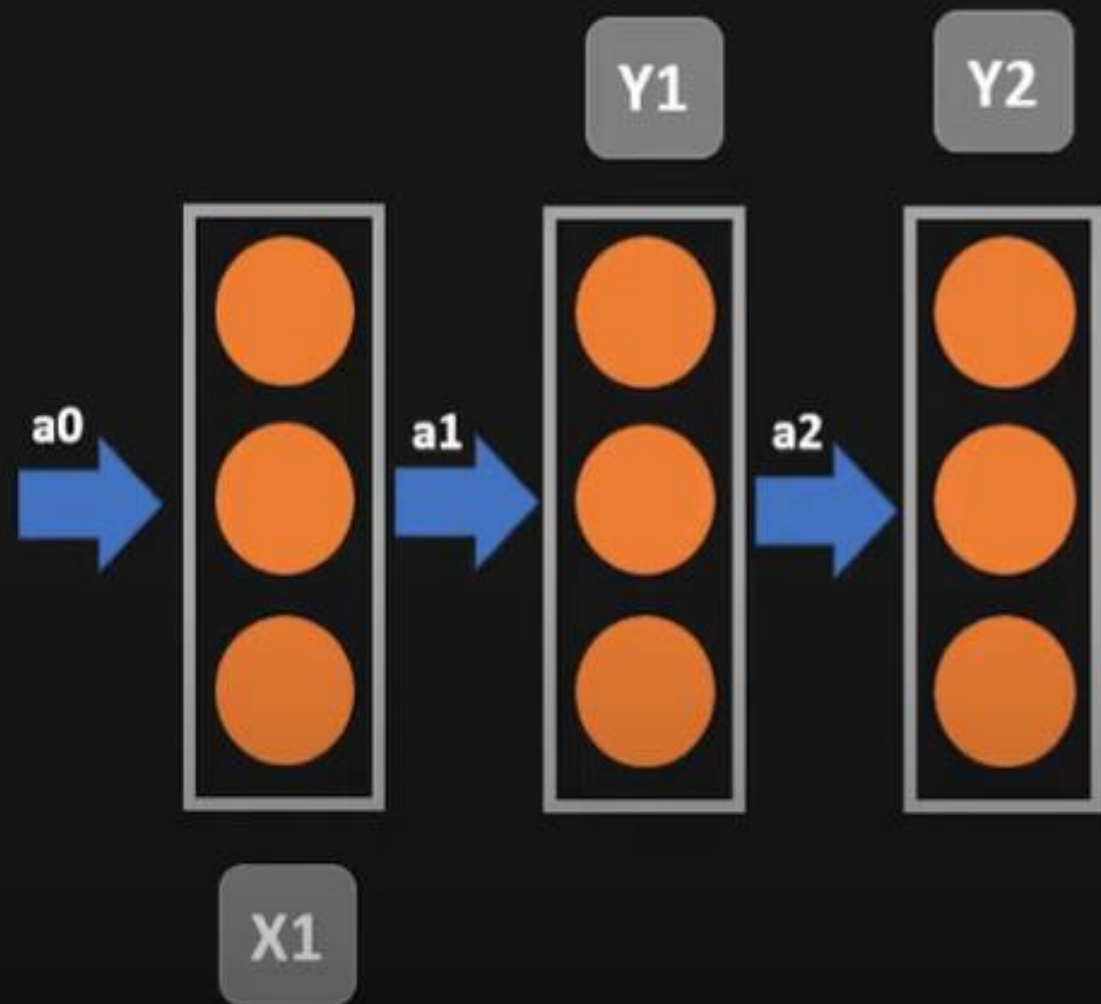


One To Many

In this type of RNN, there is one input and many outputs associated with it. One of the most used examples of this network is Image captioning where given an image we predict a sentence having Multiple words.



One to Many

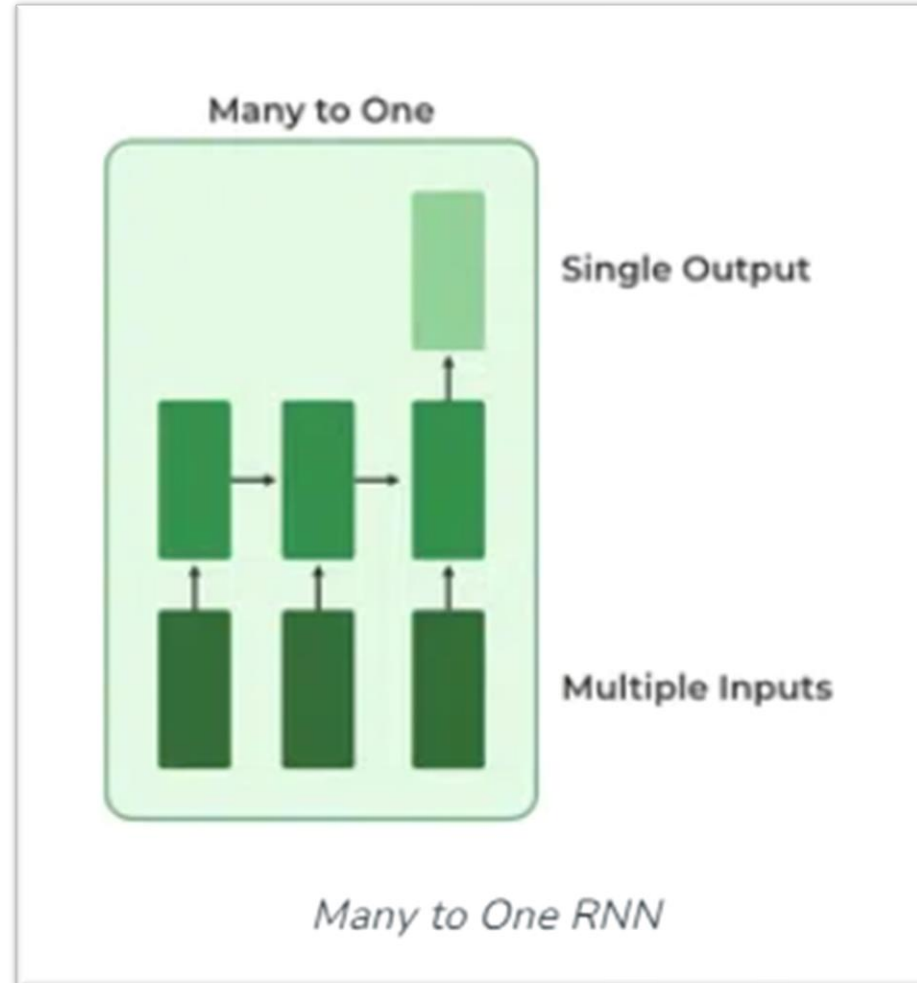


Music Generation

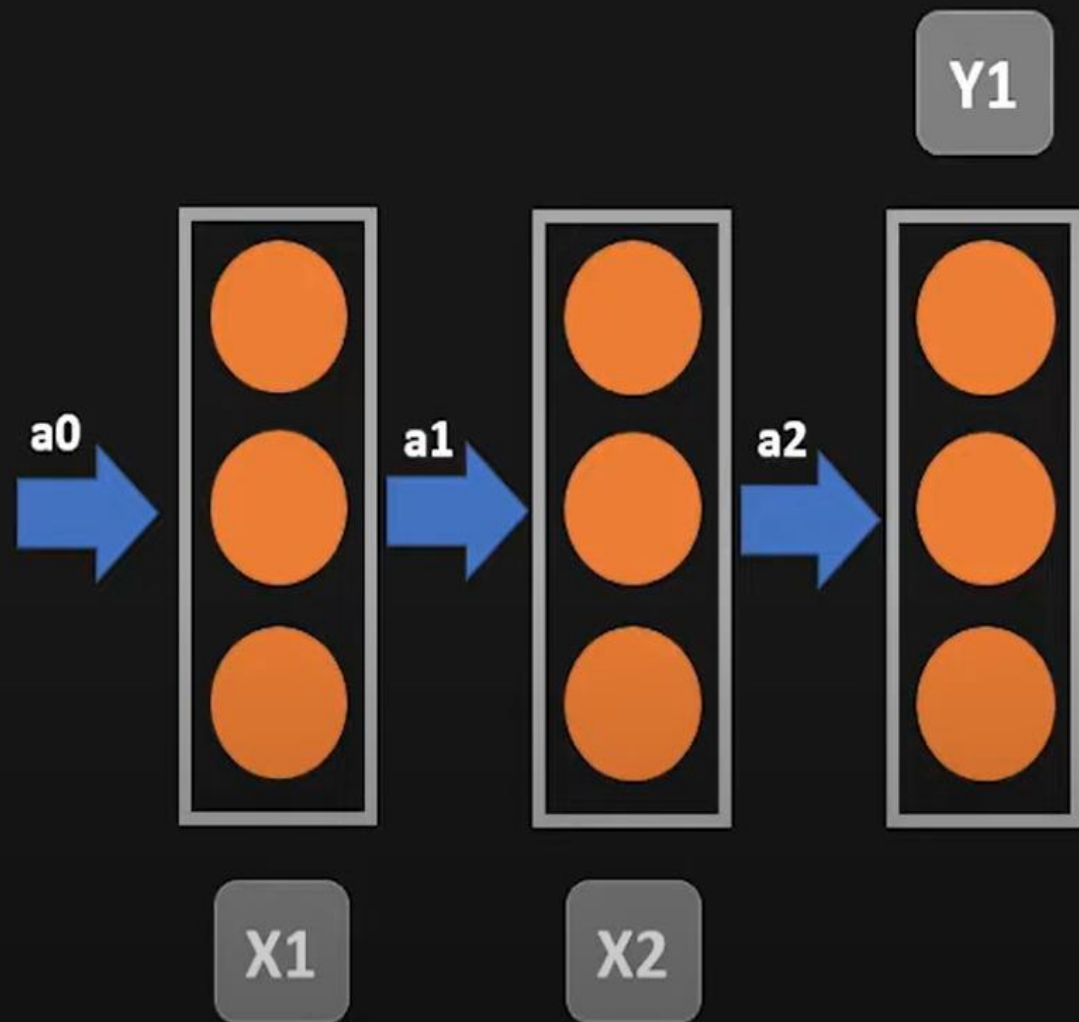


Many to One

In this type of network, Many inputs are fed to the network at several states of the network generating only one output. This type of network is used in the problems like sentimental analysis. Where we give multiple words as input and predict only the sentiment of the sentence as output.

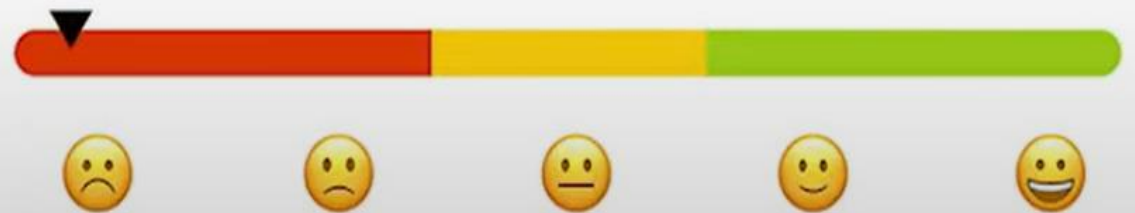


Many to One



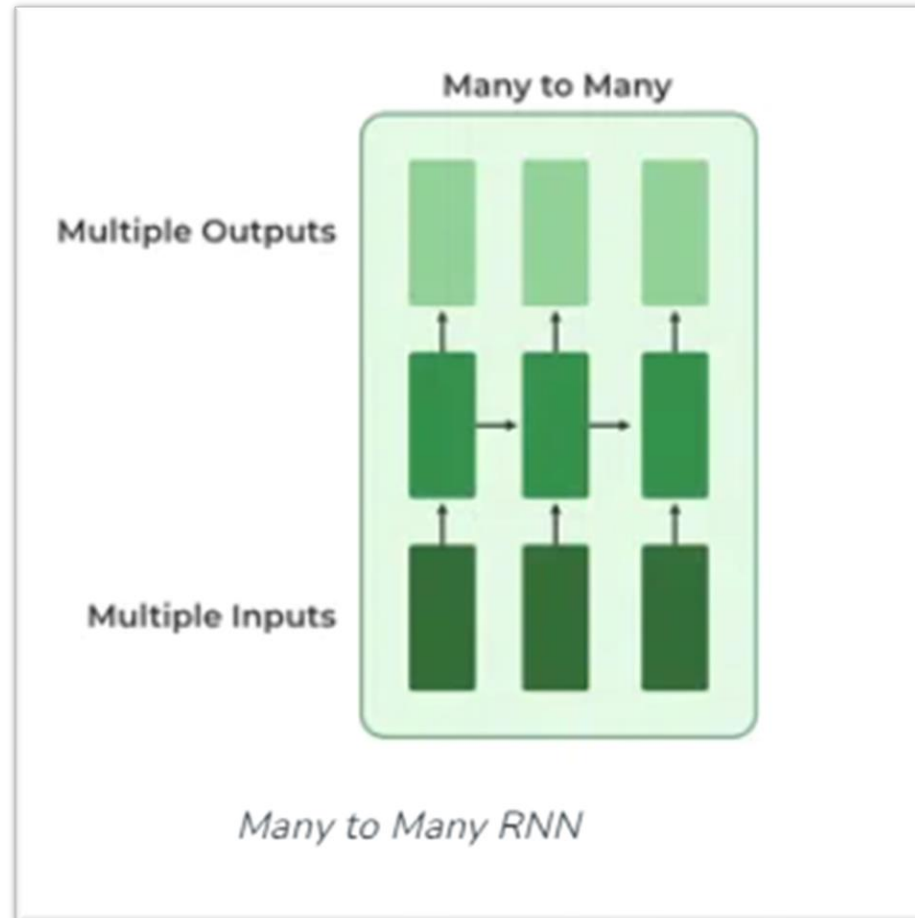
“I DON'T LIKE IT”

Their customer service is a total nightmare.

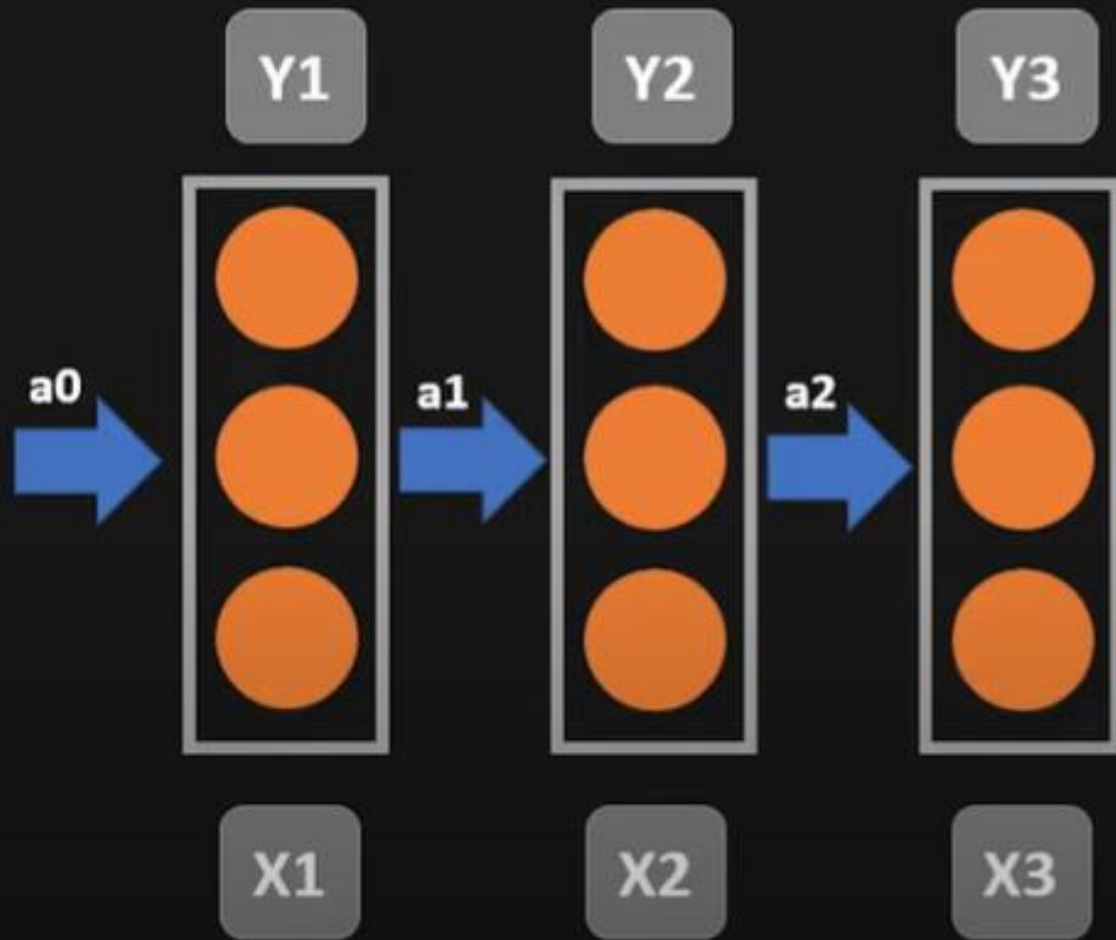


Many to Many

In this type of neural network, there are multiple inputs and multiple outputs corresponding to a problem. One Example of this Problem will be language translation. In language translation, we provide multiple words from one language as input and predict multiple words from the second language as output.



Many to Many



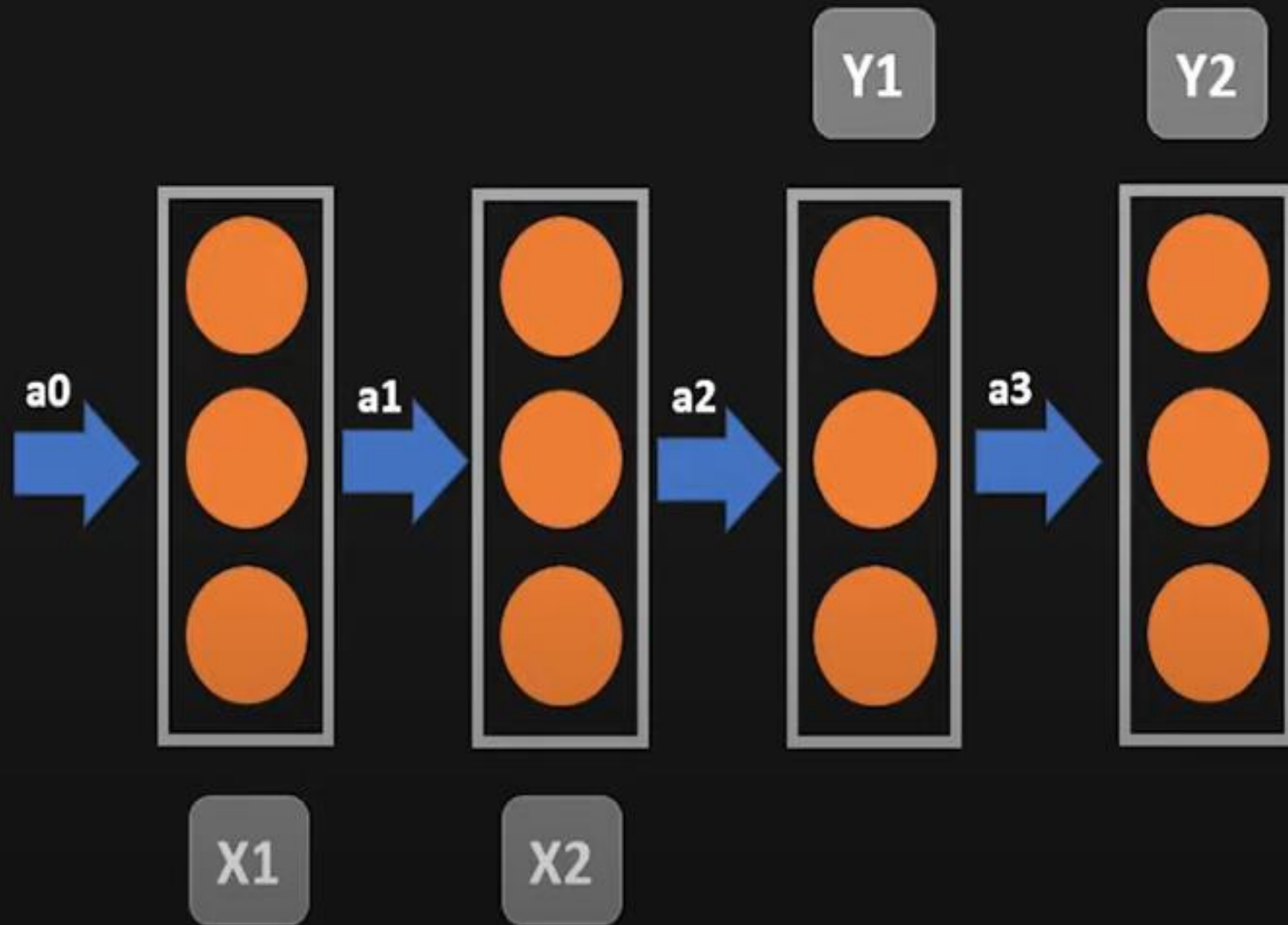
**DNA SEQUENCE
ANALYSIS**

AGCCCCCTGTGAG



AGC**CC**CTGTGAG

Many to Many



मुझे नीला रंग पसंद है



I like blue colour

Disadvantages of Recurrent Neural Network

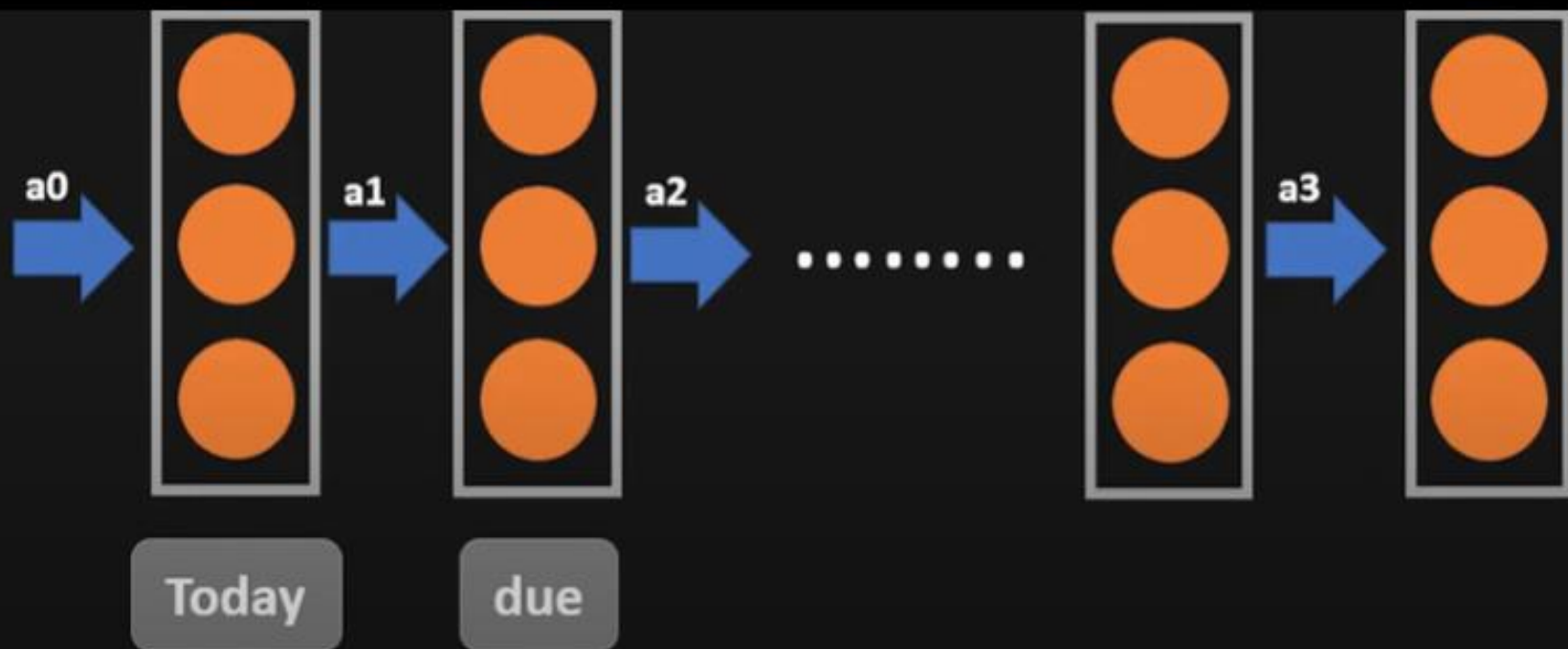
- ❑ Vanishing and Exploding Gradient Problems: RNNs are prone to vanishing and exploding gradient problems, which can occur during training when gradients either become too small or too large, leading to difficulties in learning long-range dependencies.
- ❑ Difficulty in Capturing Long-Term Dependencies: Standard RNN architectures may struggle to capture long-term dependencies in sequences due to the vanishing gradient problem. While architectures like LSTM and GRU mitigate this issue to some extent, they may still encounter challenges in modeling very long sequences.
- ❑ Computationally Intensive: Training RNNs can be computationally intensive, especially for long sequences and large datasets. This is due to the recurrent nature of the network, which requires multiple iterations over the sequence during both forward and backward passes.

- ❑ **Limited Context Window:** RNNs have a limited context window, meaning they can only consider a fixed number of previous time steps when making predictions. This limitation may hinder their performance in tasks requiring a broader context, such as document-level sentiment analysis.
- ❑ **Difficulty in Parallelization:** RNNs are inherently sequential in nature, which makes them challenging to parallelize effectively. This can lead to slower training times compared to feedforward neural networks and other parallelizable architectures.

VANISHING GRADIENT

Today, due to my current job situation and family conditions, I need to take a loan.

Last Year, due to my current job situation and family conditions, I had to take a loan.



Variation Of Recurrent Neural Network (RNN)

To overcome the problems like vanishing gradient and exploding gradient descent several new advanced versions of RNNs are formed some of these are as;

1. Bidirectional Neural Network (BiNN)

2. Long Short-Term Memory (LSTM)

1) Bidirectional Neural Network (BiNN)

A BiNN is a variation of a Recurrent Neural Network in which the input information flows in both direction and then the output of both direction are combined to produce the input. BiNN is useful in situations when the context of the input is more important such as Nlp tasks and Time-series analysis problems.

2) Long Short-Term Memory (LSTM)

Long Short-Term Memory works on the read-write-and-forget principle where given the input information network reads and writes the most useful information from the data and it forgets about the information which is not important in predicting the output. For doing this three new gates are introduced in the RNN. In this way, only the selected information is passed through the network.

Introduction to LSTM

LSTM networks are an extension of recurrent neural networks (RNN) mainly introduced to handle situations where RNNs fail.

- It fails to store information for a longer period of time. At times, a reference to certain information stored quite a long time ago is required to predict the current output. But RNNs are absolutely incapable of handling such “long-term dependencies”.
- There is no finer control over which part of the context needs to be carried forward and how much of the past needs to be ‘forgotten’.
- Other issues with RNNs are exploding and vanishing gradients (explained later) which occur during the training process of a network through backtracking.

Thus, Long Short-Term Memory (LSTM) was brought into the picture. It has been so designed that the vanishing gradient problem is almost completely removed, while the training model is left unaltered. Long-time lags in certain problems are bridged using LSTMs which also handle noise, distributed representations, and continuous values.

LSTMs provide us with a large range of parameters such as learning rates, and input and output biases.

Applications of LSTM Networks

LSTM models need to be trained with a training dataset prior to their employment in real-world applications. Some of the most demanding applications are discussed below:

1. Language modeling or text generation, involves the computation of words when a sequence of words is fed as input. Language models can be operated at the character level, n-gram level, sentence level, or even paragraph level.

2. Image processing involves performing an analysis of a picture and concluding its result into a sentence. For this, it's required to have a dataset comprising a good amount of pictures with their corresponding descriptive captions. A model that has already been trained is used to predict features of images present in the dataset. This is photo data. The dataset is then processed in such a way that only the words that are most suggestive are present in it. Speech and Handwriting Recognition.

3. Music generation is quite similar to that of text generation where LSTMs predict musical notes instead of text by analyzing a combination of given notes fed as input.

4. Language Translation involves mapping a sequence in one language to a sequence in another language. Similar to image processing, a dataset, containing phrases and their translations, is first cleaned and only a part of it is used to train the model. An encoder-decoder LSTM model is used which first converts the input sequence to its vector representation (encoding) and then outputs it to its translated version.

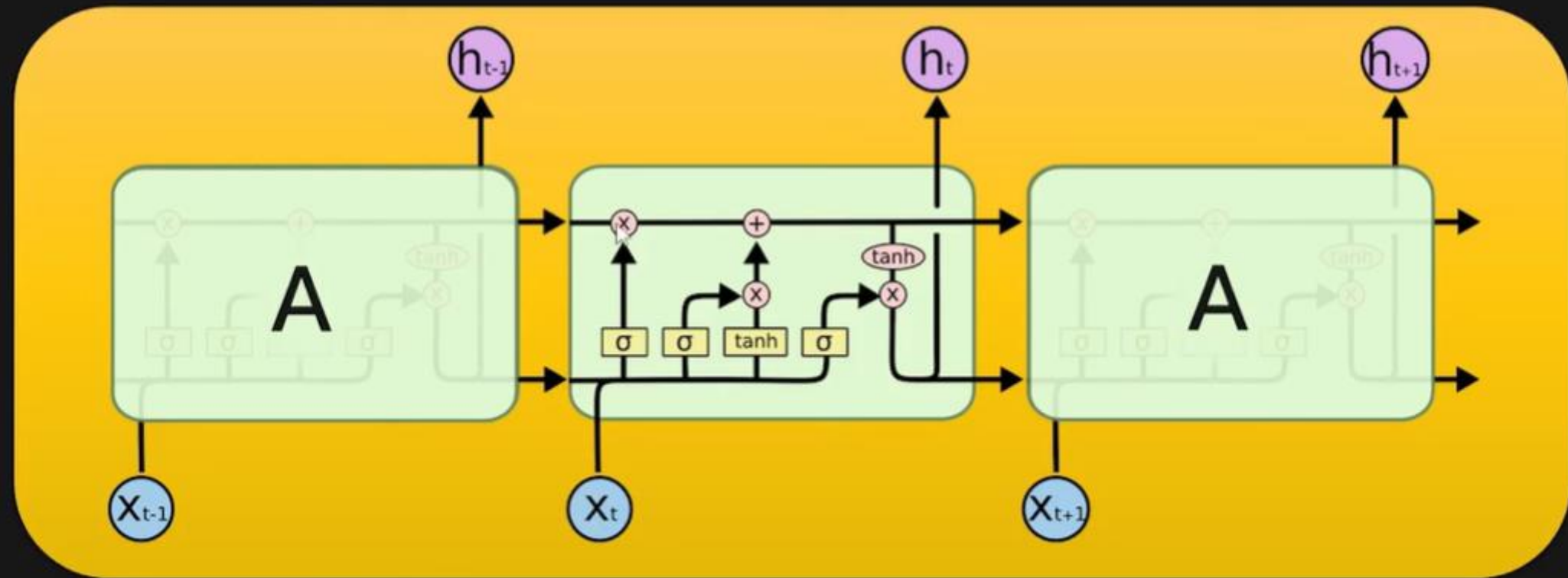
Long Short Term Memory

Today, due to my current job situation and family conditions, I.....

Today, due to my current job situation and family conditions, I need to take a loan.

Long Short Term Memory

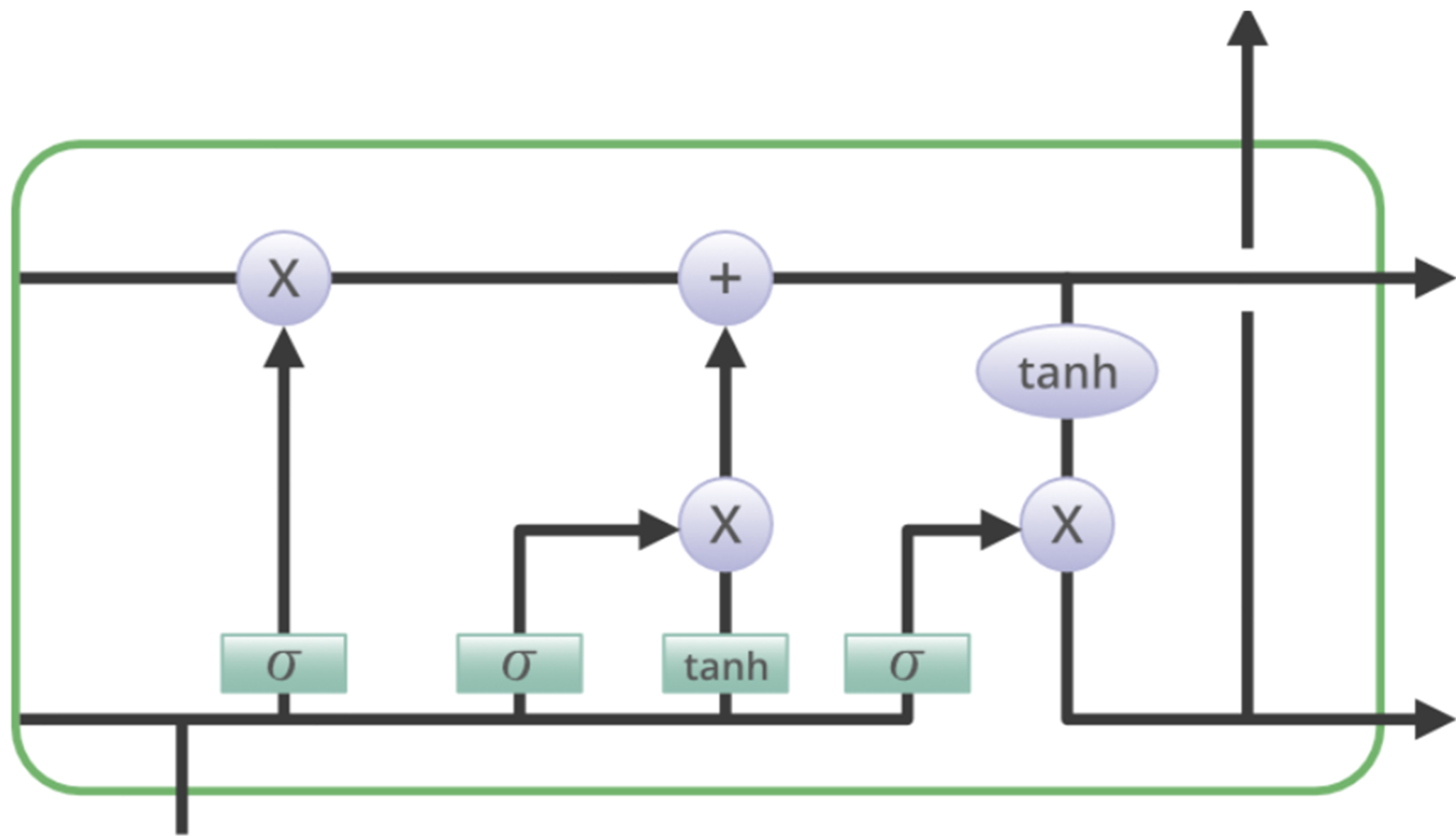
Today, due to my current job situation and family conditions, I....



Structure of LSTM

The basic difference between the architectures of RNNs and LSTMs is that the hidden layer of LSTM is a gated unit or gated cell. It consists of four layers that interact with one another in a way to produce the output of that cell along with the cell state. These two things are then passed onto the next hidden layer. Unlike RNNs which have got only a single neural net layer of tanh, LSTMs comprise three logistic sigmoid gates and one tanh layer. Gates have been introduced in order to limit the information that is passed through the cell. They determine which part of the information will be needed by the next cell and which part is to be discarded. The output is usually in the range of 0-1 where '0' means 'reject all' and '1' means 'include all'.

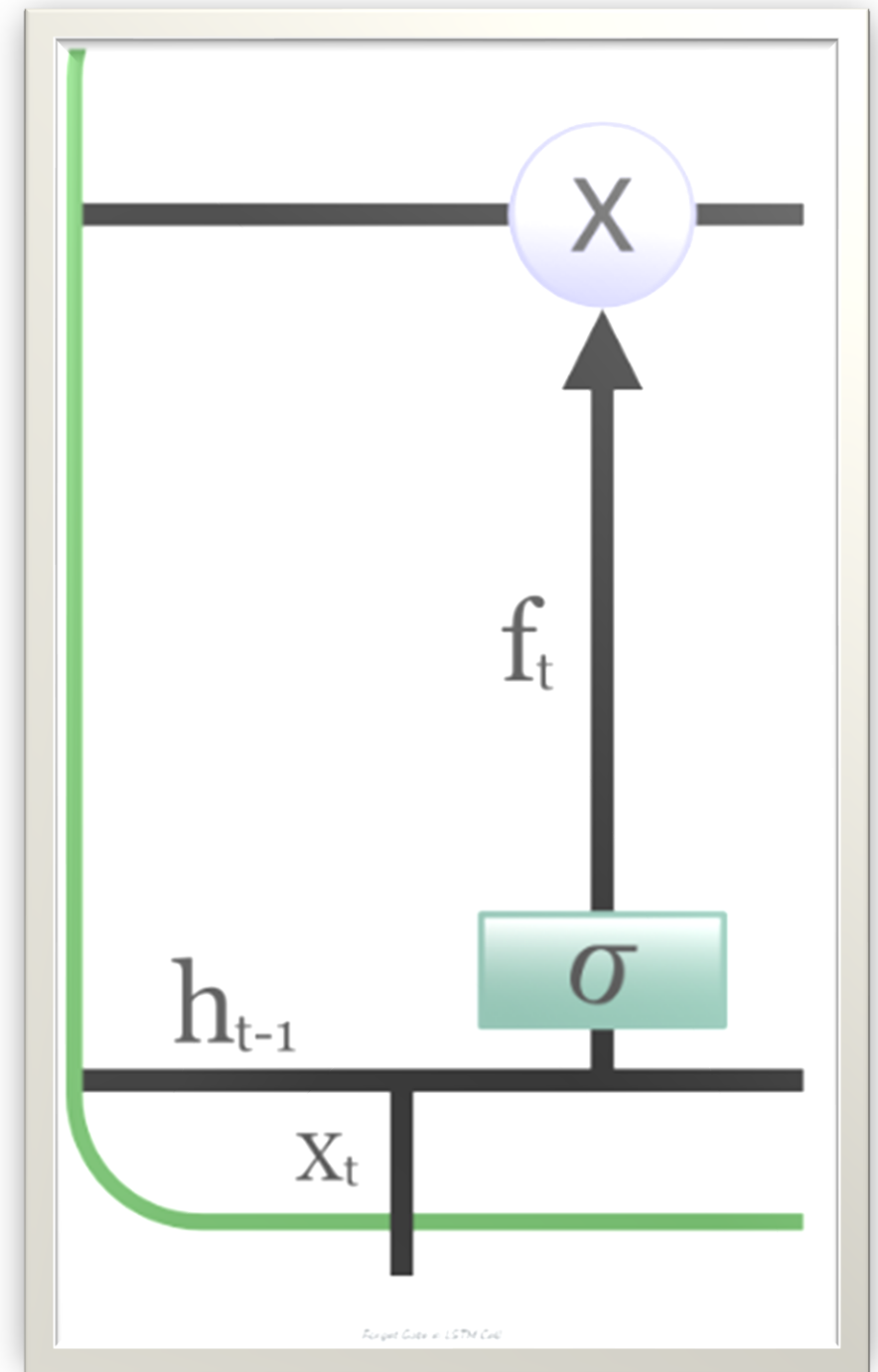
Information is retained by the cells and the memory manipulations are done by the gates.



Structure of an LSTM Network

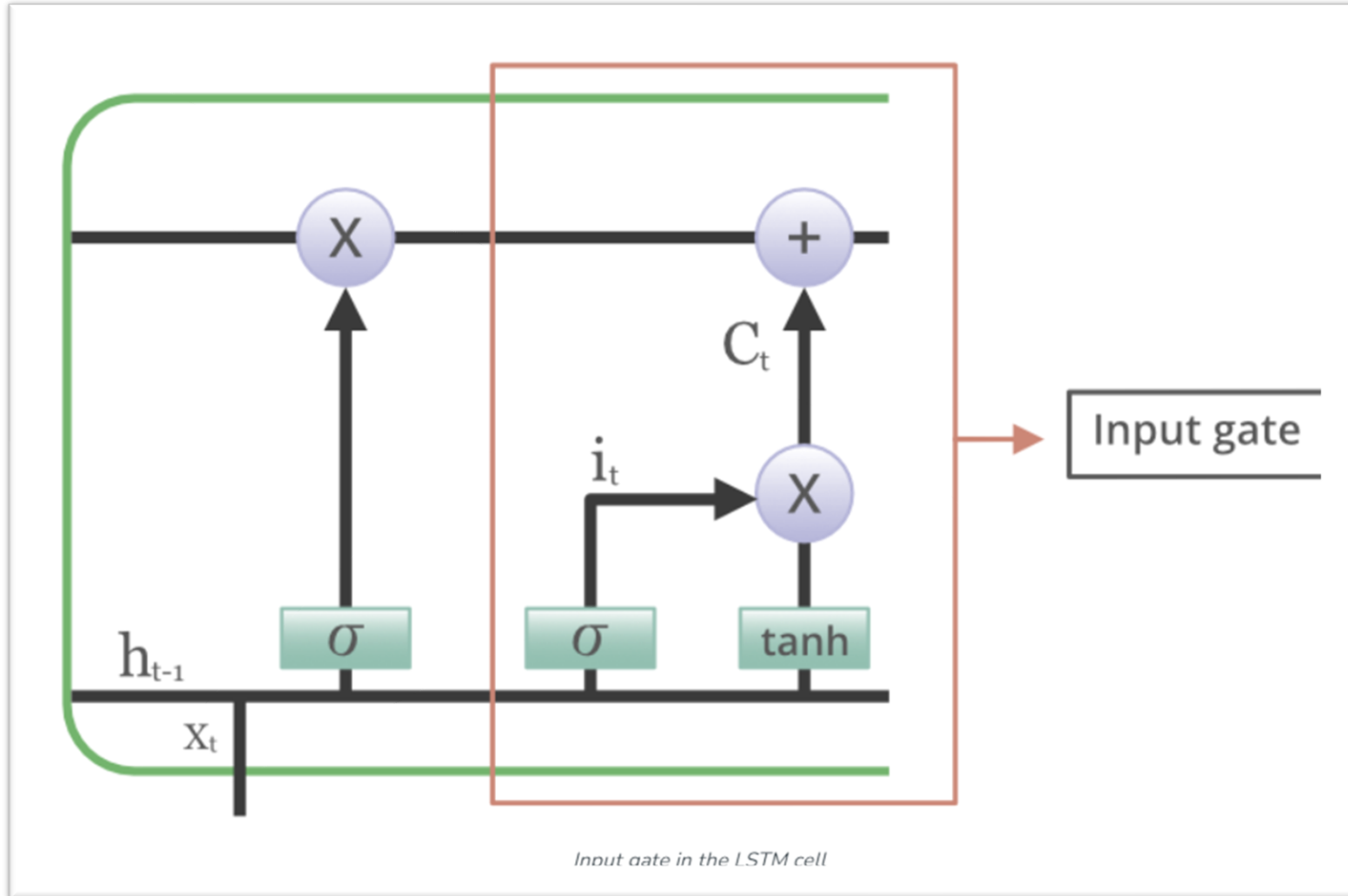
Forget Gate

The information that is no longer useful in the cell state is removed with the forget gate. Two inputs x_t (input at the particular time) and h_{t-1} (previous cell output) are fed to the gate and multiplied with weight matrices followed by the addition of bias. The resultant is passed through an activation function which gives a binary output. If for a particular cell state, the output is 0, the piece of information is forgotten and for output 1, the information is retained for future use.



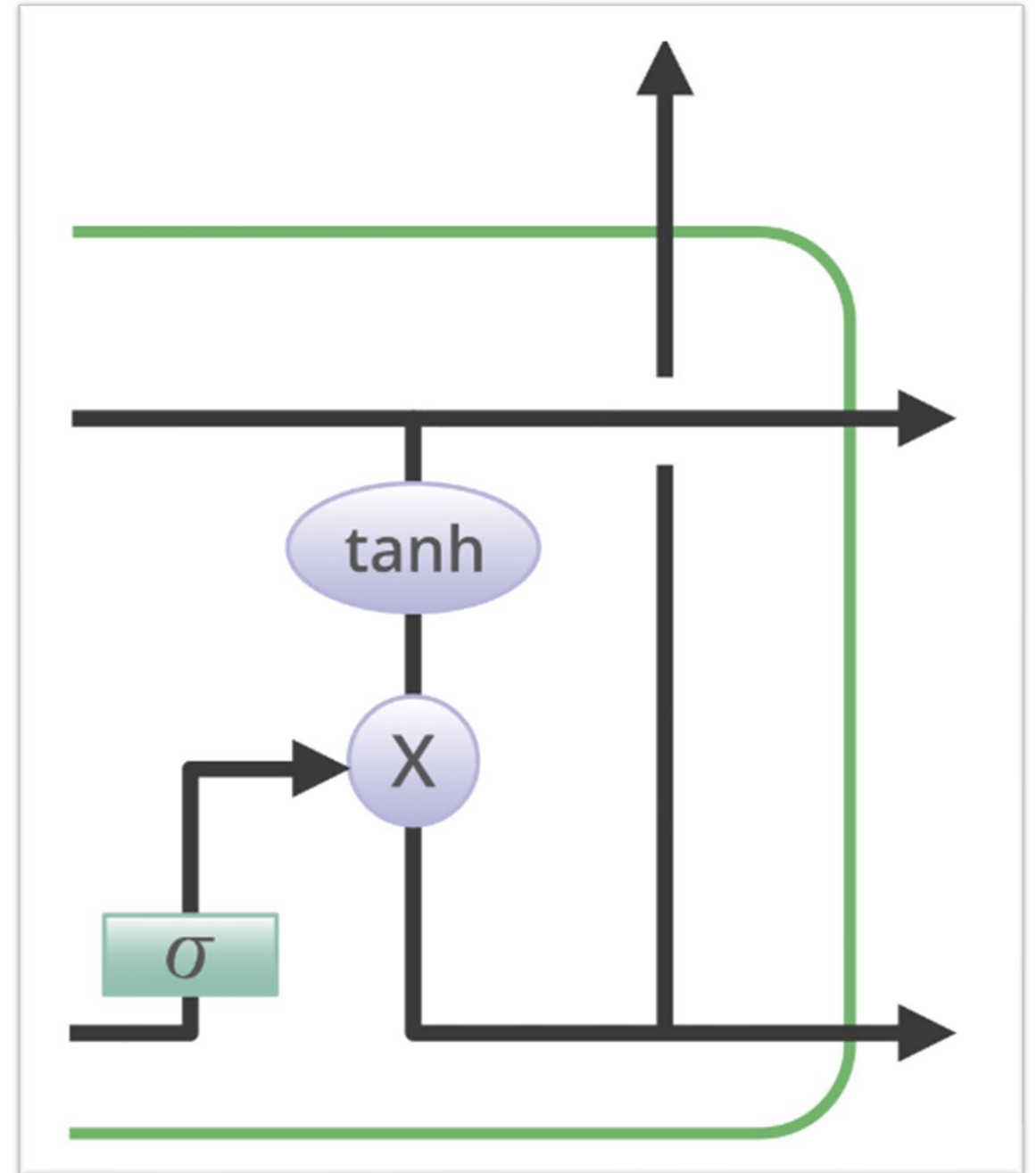
Input gate

The addition of useful information to the cell state is done by the input gate. First, the information is regulated using the sigmoid function and filter the values to be remembered similar to the forget gate using inputs h_{t-1} and x_t . Then, a vector is created using the tanh function that gives an output from -1 to +1, which contains all the possible values from h_{t-1} and x_t . At last, the values of the vector and the regulated values are multiplied to obtain useful information.



Output gate

The task of extracting useful information from the current cell state to be presented as output is done by the output gate. First, a vector is generated by applying the $\tanh$ function on the cell. Then, the information is regulated using the sigmoid function and filtered by the values to be remembered using inputs h_{t-1} and x_t . At last, the values of the vector and the regulated values are multiplied to be sent as an output and input to the next cell.



Rohit love apples



vs



Name Entity Recognition

Rohit, loves apple, it keeps him healthy

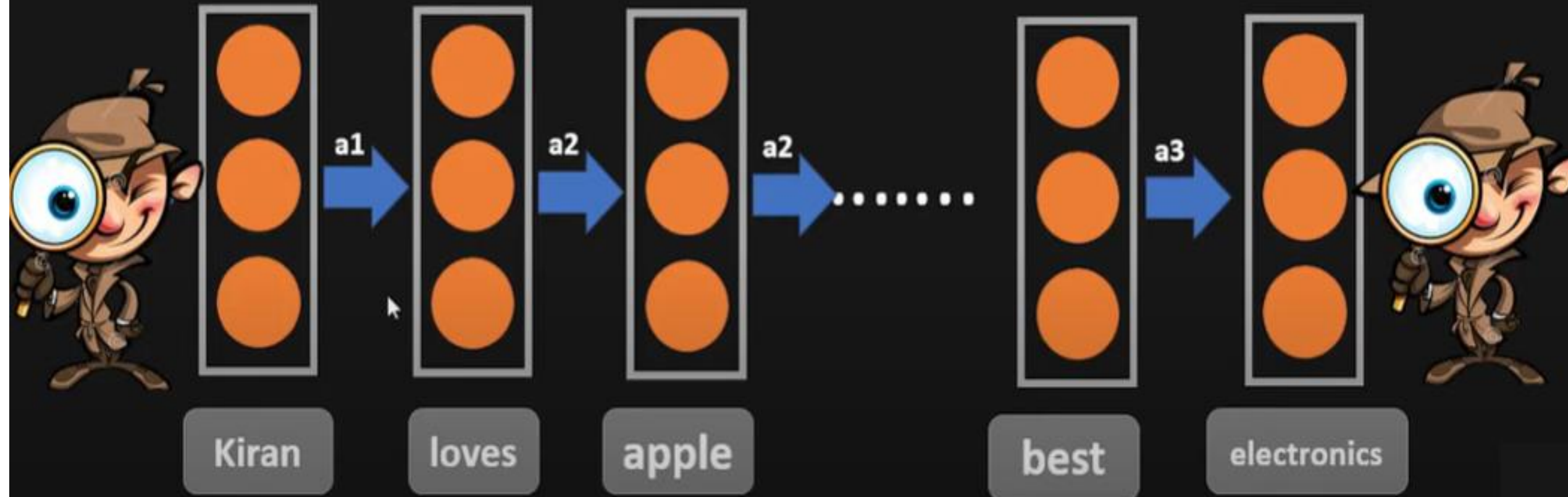


Name Entity Recognition

Rohit loves **apple**, the company products is best in **electronics**



Bidirectional RNN



Bidirectional RNN

